Desarrollo del Sistemas de Gestión - Prototipo funcional de la aplicación cliente-servidor en Go

CHAT EMPRESARIAL

Darwin Alcarraz, José Almeida, Cayetano Córdova

Facultad de Ciencias Técnicas, Universidad Internacional Del Ecuador

03-SIN-A: Programación Orientada a Objetos

Ing. Ivan Galo Reyes Chacón

8 de noviembre de 2026

1. Objetivo del Prototipo

Construir una primera versión ejecutable del chat empresarial, con un servidor y un cliente en Go que se comunican por HTTP, para enviar mensajes y consultar el historial, aplicando los conceptos aprendidos en clase como “structs”, “métodos”, “encapsulamiento”, etc, como base de las siguientes fases.

2. Funcionalidades implementadas.

El prototipo permite a los usuarios iniciar sesión a través de un cliente de consola usando cuentas de prueba (ana, luis o marta). El servidor verifica el registro del usuario y, al cerrar sesión, el cliente elimina el token y retorna a la pantalla de inicio. Una vez dentro, los usuarios pueden enviar y recibir mensajes de texto, que son verificados por el cliente cada 3 segundos sin bloquear el menú, gracias a un manejo en goroutine. Además, los usuarios pueden consultar un historial de conversación, que se organiza en páginas de 50 mensajes por fecha. Los mensajes se almacenan en un archivo JSON, asegurando su disponibilidad tras el reinicio del servidor. La configuración de la dirección IP y el puerto es posible mediante parámetros de línea de comandos en ambos, cliente y servidor.

El prototipo valida los datos y proporciona mensajes claros ante errores comunes, como mensajes vacíos, excesivamente largos (más de 500 caracteres), mensajes destinados al propio usuario, destinatarios inexistentes, tokens inválidos, parámetros incorrectos y opciones inexistentes en el menú. También gestiona fallos en los datos (archivos dañados o errores de escritura) y problemas de conexión (servidor apagado) sin recurrir a panic. Se han implementado pruebas automáticas a lo largo de los seis paquetes del proyecto. Sin embargo, la autenticación real con contraseña, hash y vencimiento de token, así como el registro de estadísticas de uso

(RF-05), no están incluidos en esta versión y están pendientes para el servicio FastAPI de las semanas 11 y 12.

3. **Requisitos atendidos y requisitos pendientes.**

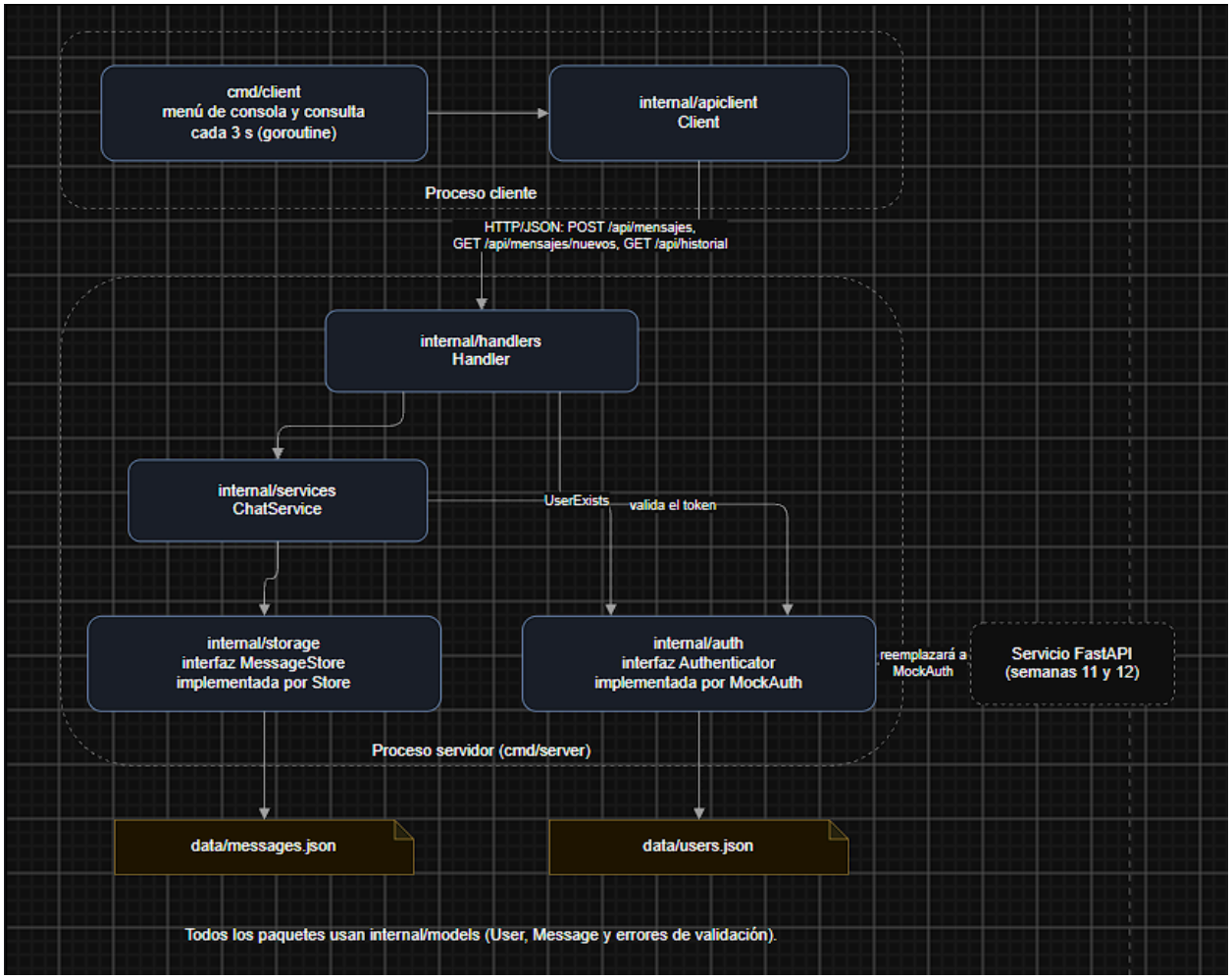
Requisitos atendidos. Se cumplen el envío y recepción de mensajes (RF-02) y el historial por páginas de 50 mensajes (RF-03). También se cumplen los requisitos no funcionales: goroutines en el cliente (RNF-01), token en cada petición (RNF-02), uso de la librería estándar de Go, probado en Linux (RNF-03), y comunicación por HTTP con JSON (RNF-05).

Requisitos parciales. La autenticación (RF-01) y el control de sesiones (RF-04) están simulados: el cliente inicia y cierra sesión con un token de prueba. El login real, con contraseña, hash y vencimiento del token, corresponde al servicio FastAPI de las semanas 11 y 12.

Requisitos pendientes. El registro de datos de uso (RF-05): los mensajes se guardan con usuario, fecha y hora, pero aún no se registran las demás acciones. El hash de contraseñas (RNF-04) no aplica en Go, porque es del servicio FastAPI.

No se modificó ningún requisito aprobado, por lo que la matriz de trazabilidad sigue vigente.

4. **Diagrama actualizado de componentes, módulos o estructuras.**



5. Explicación de la comunicación cliente-servidor.

El cliente y el servidor son dos procesos independientes que se comunican por HTTP con datos en JSON. El servidor escucha en una IP y un puerto configurables (por defecto "127.0.0.1:8080"), y el cliente se conecta a esa dirección.

Cada petición lleva el token en el encabezado "Authorization". El servidor lo valida antes de procesarla y responde 401 si no es válido. El cliente usa tres endpoints: "POST /api/mensajes" para enviar, "GET /api/mensajes/nuevos" para recibir y "GET /api/historial" para consultar una conversación.

Al enviar un mensaje, el servidor lo valida, comprueba que el destinatario exista y lo guarda en JSON. Si algo falla, responde con un código de error (400, 401 o 404) y un mensaje claro.

El servidor no avisa al destinatario: su cliente consulta cada 3 segundos, en una goroutine, si hay mensajes nuevos, así el menú no se congela. Si el servidor no está disponible, el cliente muestra un aviso y no se cierra.

6. Descripción de las principales estructuras, constructores y métodos.

El prototipo se organiza en siete estructuras, cada una con su constructor y sus métodos.

"User" representa a un empleado. Sus campos no son exportados, así que solo se usan mediante métodos: "Username()", "DisplayName()" y "SetDisplayName()", que usa receptor por puntero porque modifica el estado. Su constructor, "NewUser", valida el nombre de usuario (3 a 20 caracteres) y garantiza que nunca exista un usuario inválido.

"Message" representa un mensaje entre dos usuarios. Su constructor, "NewMessage", valida que el texto no esté vacío, que no supere los 500 caracteres y que el remitente no sea el destinatario, y devuelve un "error" si algo falla. Tiene el método "IsBetween(a, b)", que indica si pertenece a la conversación entre dos usuarios. Sus campos son exportados, con etiquetas JSON, porque viaja por la red y se guarda en un archivo.

"Store" guarda los mensajes en memoria y en "messages.json". Se crea con "NewStore(path)", que carga el archivo si existe. Sus métodos son "Save", que asigna el ID y escribe el archivo, "Conversation", que devuelve los mensajes entre dos usuarios, y "Pending", que devuelve los mensajes nuevos de un usuario. Sus campos son privados y usa un "sync.Mutex" para que dos peticiones simultáneas no dañen los datos.

"MockAuth" es el autenticador temporal. Se crea con "NewMockAuth(path)", que carga los usuarios de prueba, y tiene los métodos "Validate", que comprueba el token y devuelve el usuario, y "UserExists".

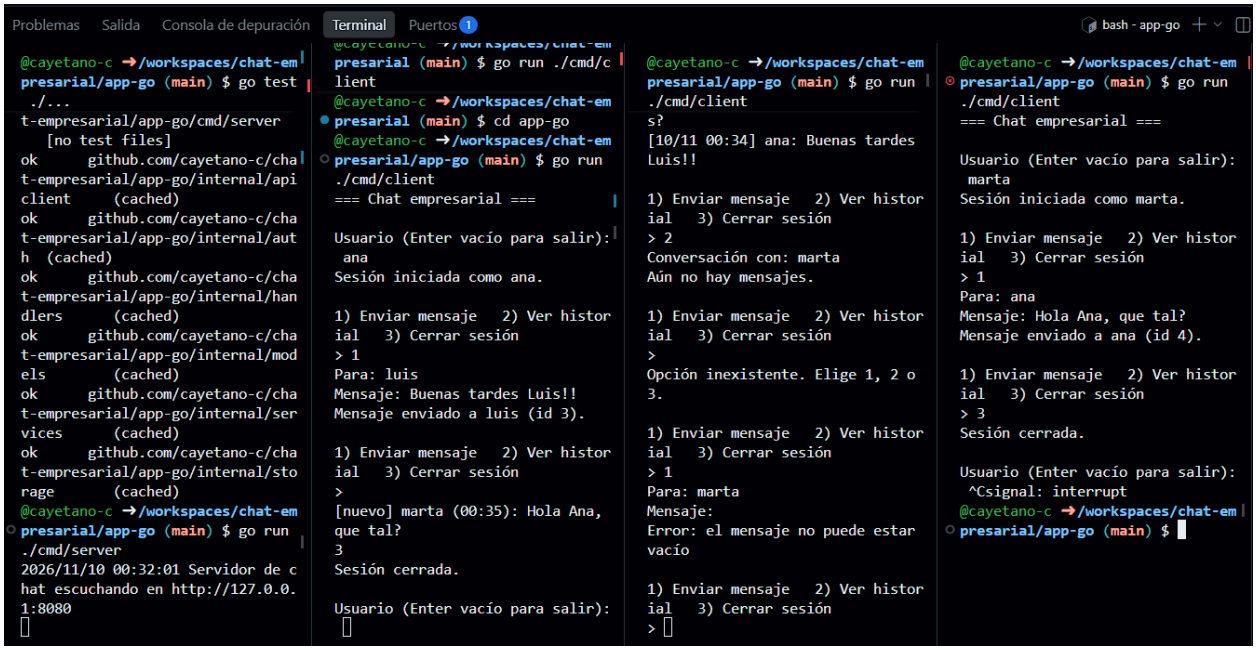
"ChatService" contiene la lógica del chat. Se crea con "NewChatService(store, users)" y tiene "SendMessage", "NewMessages" e "History" (con paginación de 50). Es un caso de composición: contiene un almacén y un autenticador, y no conoce nada de HTTP.

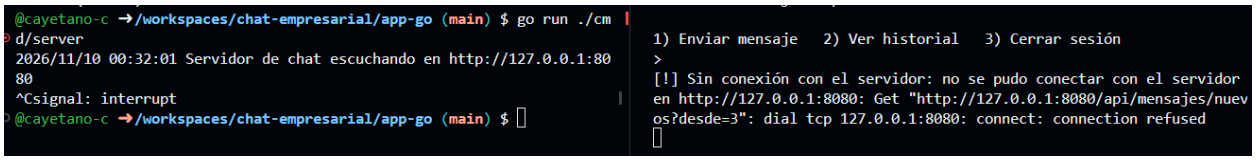
"Handler" recibe las peticiones HTTP. Se crea con "New(svc, users)". Su método "Routes" registra los endpoints, y "sendMessage", "newMessages" e "history" los atienden. También valida el token y traduce los errores a códigos HTTP. Contiene al servicio, como otro caso de composición.

"Client" es el cliente del lado del usuario. Se crea con "New(baseURL)" y guarda la sesión. Sus métodos son "Login", "Logout", "Send", "NewMessages" e "History". Los errores que responde el servidor se devuelven como "APIError", para distinguirlos de los fallos de conexión.

Interfaces. Se usan dos, ambas pequeñas. "MessageStore" ("Save", "Conversation", "Pending") separa la lógica del chat de la forma de guardar los datos. "Authenticator" ("Validate", "UserExists") permite reemplazar la autenticación de prueba por el servicio FastAPI sin reescribir el chat. "Store" y "MockAuth" cumplen cada una su interfaz de forma implícita, sin declararlo.

7. **Evidencias de ejecución.**





8. Limitaciones actuales del prototipo.

La autenticación es una simulación: no hay contraseñas ni vencimiento de tokens, y se reemplazará con el servicio FastAPI en las semanas 11 y 12. Tampoco se registra aún el uso del sistema (RF-05), más allá de los mensajes guardados. Los mensajes viven en un único archivo JSON que se reescribe completo en cada envío, algo suficiente para el prototipo pero no para un volumen grande. Los mensajes nuevos llegan consultando al servidor cada 3 segundos, no en tiempo real. La comunicación es HTTP sin cifrar, pensada para un entorno local o de laboratorio. Quedan fuera del alcance los chats de grupo, el envío de archivos y el registro de usuarios desde la aplicación.

9. Enlace al repositorio Git.

https://github.com/cayetano-c/chat-empresarial

10. Enlace al video demostrativo.

https://drive.google.com/file/d/1ZV4PqNrcFLGNOaluqNKmE9oRu0QNwYyP/view?usp=sharing